

Теорія алгоритмів як самостійна наука з'явилася в 30-х ХХ ст, завдяки цій теорії знаходять свої такі фундаментальні поняття як алгоритм, доказовість, складність. Термін алгоритм походить від імені вченого Аль-Хорезмі Мухаммед ібн Мусі з Хорезмі (825 р н.е.) цей вчений вперше виклав десяткову систему числення та правила виконання простих арифметичних дій перед елементами цієї системи. Як науковий термін алгоритм спочатку означав лише правила арифметичних операцій над цілими числами та простими дробами. Потім цей термін набув ширше значення яке включало точні приписи, правила виконання деяких простих операцій, визначення процесу перетворення вихідних даних на результат. В даному вигляді поняття алгоритму сформувалося на початку ХХ-го століття завдяки роботам таких вчених: Д.Гліберт, С.Клаіні, А.Черч, Е.Пост, А.Тюрінг, Н.Віннерю. Одним із найстаріших чисельних алгоритмів вважається алгоритм Евкліда 3 ст д.е..

Сучасна теорія алгоритмів почалася з робіт німецького математика К. Геделя(1931) У яких було показано існування завдань не вирішуваних у межах своєї формальної несуперечливо системи аксіом. Пресі фундаментальні роботи з теорії алгоритмів з'явилися у 1936р було запропоновано машину Тінлінга та лямбда числення Чорча, Ці машини були еквівалентним нормалізаціями алгоритму.

Не існує ... так як це поняття є фундаментальним і не має однозначного визначення. Загальними словами Алгоритм представляє собою систему правил, описує послідовність дій, які потрібно виконати щоб вирішити завдання. Найбільш поширеним є визначення за кнотом алгоритм — це кінцева множина правил, яка задає послідовність операцій, для вирішення конкретної множини задач, та володіє п'ятьма ознаками: 1)ефективність 2) скінченність 3) визначеність 4)ввід 5)вивід де є кнуд. Ефективність алгоритму— властивість алгоритму що пов'язана з обчислювальними ресурсами що потрібна алгоритму. Скінченність — кожна з дій та весь алгоритм загалом обов'язково завершиться Визначеність — це властивість яка вказує алгоритму, що кожен його крок має бути суворо визначеним тобто різні тлумачення мають бути виключення. Строго визначається і порядок кроків.

Формальні властивості алгоритмів

Алгоритм володіє такими формальними властивостями: дискретність, визначеність, результативність, масовість. Дискретність означає що виконання алгоритму розвивається на послідовність виконаних дій — кроків. Кожна дія має бути завершена виконавцем перш ніж він перейде до виконання наступного кроку. Значення величин у кожному кроці алгоритму виходять за певними правилами, і значень величини визначених на попередньому кроці. Під визначеністю розуміється така чіткість кожного правила алгоритму, що значення величин одержувані на якомусь кроці однозначно визначаються на значенні величин отримуваними на попередньому кроці і при цьому відомо який крок буде виконаний наступним. Це означає також що виконавець (компютер) повинен мати можливість виконати кожен команду алгоритму у суворій відповідності до її призначення. Результативність алгоритму передбачає що його виконання зводиться до виконання кінцевого числа дій і завжди призводить до деякого результату. Рішенням може бути і відсутність рішення завдання.Під масовістю розуміється що алгоритм розв'язання задачі розробляється у загальному вигляді так, щоб його можна було застосувати для цілого класу задач , що відрізняються лише наборами вихідних даних, для цього вихідні дані можуть вибиратися з деякої області, яка називається область застосування алгоритму, при цьому вихідні дані можуть вибиратися з деякої області яка називається областю застосування алгоритму, в якості масовості укладена основна практична цінність алгоритмів.

Алгоритмізація

Алгоритмізація — процес отримання та формування алгоритму прийнято виділяти загальну(абстрактну) , та прикладну(структурну) теорію алгоритмів. Загальна теорія алгоритмів

розглядає питання принципової здійсненності алгоритму без урахування обмежень, що накладаються технічною системою на його реалізацію. Основна проблема загальної теорії алгоритмів це проблема алгоритмічної розв'язаності тієї чи іншої задачі чи класу завдань. При цьому під алгоритмічною роздільною здатністю розуміється можливість алгоритмізації вихідної задачі. Для складання алгоритму рішення задачі на комп'ютері, необхідно перш за все формалізувати задачу тобто подати її у вигляді послідовності математичних співвідношень, потім перетворити ці співвідношення на послідовність дій та логічних зв'язків між ними.

Способи опису алгоритмів

Будь-який алгоритм записується у вигляді певної послідовності виконання деяких дій(операцій) над вхідною інформацією. Операції поймають кодуються певними символами, що зображують арифметичні, логічні та інші операції. Залежно від символів, що використовуються можливі різні способи запису алгоритмів. Основними серед них є: словесний опис алгоритму (природною мовою). Блок-схема(графічний метод). Псевдо-код, запис якоюсь мовою програмування. Існують і інші способи: рядковий запис алгоритму, операторний спосіб (мовою операторних схем), предактивний.

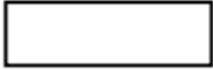
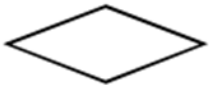
Кожен із цих способів повинен задовольняти використовуваним символам. Виключення неоднозначних тлумачень, та можливостей складання доступних для огляду алгоритмів.

Словесний опис

Словесний опис є найменш формалізованим і має найменше обмежень. В результаті до такого опису пред'являється менше вимог, що може призвести до невизначеностей. Недоліком алгоритму у вербальній формі є те, що такий алгоритм може виявитись дуже об'ємним та важким для сприйняття людиною.

Графічний метод (блок-схеми)

Графічна блок-схема алгоритму — графічне зображення алгоритму із застосуванням спеціальних блоків, що мають необхідні словесні пояснення та лінії зв'язку кожен етап алгоритму представляється у вигляді геометричної фігури (блоку), що має певну форму в залежності від характеру операції. Блоки на схемі з'єднуються стрілками (лініями зв'язку), які визначають послідовність виконання операції, та утворюють логічну структуру алгоритму. Основні блоки графічної схеми можна поділити на такі схеми:

Позначення	Назва	Опис
	блок пуск-зупинка	визначає початок та кінець алгоритму (для блоку пуск (початок) - визначений тільки один вихід, для блоку зупинка (кінець) – тільки вхід);
	блок введення-виведення	визначає введення інформації в програму або виведення на пристрій
	блок процес	визначає зміну значення, форми уявлення або розташування даних
	блок перевірки умови	визначає подальші кроки виконання алгоритму в залежності від виконання умови

Весь алгоритм представляє лінійну послідовність базових структур.

Псевдо-код

Псевдо-код це досить нормальний але не дуже вимогливий до дрібниць інструмент для викладення думок не пов'язаний із конкретною мовою програмування. Псевдо-код займає проміжне положення між природною мовою, та мовою програмування. Але конкретних правил для написання псевдо-коду немає. Наприклад він може бути описаний за допомогою використання суміші синтаксису python and C де вкладеність позначається за допомогою відступів та методи мають назву python style. Псевдо-код застосовують переважно для того, щоб детальніше пояснити роботу програми. Що полегшує перевірку правильності програми. Крім того псевдо-код дає розробнику велику свободу в зображенні алгоритму, потрібно тільки використовувати стандартні управляючі конструкції та правила запису.

Запис мовою програмування.

Мова програмування — знакова система призначена для опису процесів, вирішення завдань та їх реалізації з використанням обчислювальної техніки. Реалізація означає, що описи можуть бути задані комп'ютеру і однозначно ним зрозумілі. До мов програмування відносяться мови команд, або машинні мови та мови високого рівня. Машинний код — це мови найнижчого рівня, що складаються з одиниць та нулів. Він безпосередньо інструктує процесор комп'ютера які саме операції потрібно виконати і в якому порядку, для сприйняття людиною представлення складної програми на машинній мові не зручно, оту для людини були створені мови програмування високого рівня, які не збігаються з машинними. Сутність таких мов полягає в представленні програм у вигляді не тільки прийнятному для комп'ютера, а й зручному для користувача. Комфортність означає, що спосіб запису повинен відображати основні ідеї програмування, представляти програму у формі, що легко сприймається людиною і такою, що однозначно може бути перекладеною на машинну мову.

Складність алгоритму.

Складність алгоритму — кількісна характеристика, що свідчить про те, скільки часу чи який обсяг пам'яті потрібно витратити на виконання алгоритму. Зазвичай складність алгоритмів вимірюють в елементарних кроках — те скільки дій (операцій) необхідно зробити для виконання. Будь-який алгоритм включає певну кількість кроків, і не важливо на якому пристрої він буде запущений, кількість кроків залишиться незмінною. Цю ідею прийнято представляти у вигляді Big O — нотації. Big O показує, як складність алгоритму зростає зі збільшенням вхідних даних. При цьому вона завжди показує найгірший варіант розвитку подій — верхню межу. Іншими словами Big O показує верхню межу залежності між вхідними параметрами функції та кількістю операцій які виконує процесор. Вважається що в O-нотації на операцію з однією або двома змінними, на зразок таких операцій: $i++$, $a * b$, $a / 1024$, $\max(a, b)$; йде всього одна одиниця часу.

Пам'ять чи місце — обсяг оперативної пам'яті, який знадобиться алгоритму для роботи. Одна змінна — один осередок (ячейка) пам'яті, а масив із тисячі осередків — тисяча осередків пам'яті. Теоретично в алгоритмів всі осередки вважаються рівноцінними. Наприклад змінні типу `int` мають не чотири байтів, `double` не вісім, а мають одну вагу. Споживання пам'яті зазвичай називається `space complexity`, або просто `space`, рідко `memory`, алгоритми, які використовують вихідний масив, як робочий простір називають `in-place` вони споживають мало пам'яті, і створюють одиничні змінні без копій вихідного масиву і проміжних структур даних. Алгоритми, які потребують додаткової пам'яті називають `out-of-place`. Перш ніж використовувати алгоритм треба зрозуміти чи вистачить на нього пам'яті і якщо ні, пошукати менш ненажерливі альтернативи. При розрахунку Big O Notation використовують два правила:

1. Константи відкидаються нас цікавить лише частина ф-ли, яка залежить від розміру вхідних даних; простіше кажучи це саме число n його ступінь, логарифми, факторіали та експоненти де число знаходиться у ступені n .

$$O(3n) = O(n)$$

$$O(10000 n^2) = O(n^2)$$

$$O(2n * \log n) = O(n * \log n)$$

Якщо в O є сума, то ми залишаємо складник, що найшвидше зростає. Такий підхід називається асимптотичною оцінкою складності

$$O(n^2 + n) = O(n^2)$$

$$O(n^3 + 100n * \log n) = O(n^3)$$

$$O(n! + 999) = O(n!)$$

$$O(1,1^n + n^{100}) = O(1,1^n)$$

Сценарії роботи алгоритму

Кожен алгоритм має найгірший, середній і кращий сценарій роботи — залежний від того як вдало обрані вхідні дані. Часто їх називають.

Найгірший випадок (worst case) — коли вхідні дані випадають максимальних витрат часу та пам'яті. Наприклад якщо ми хочемо відсортувати масив за зростанням. То для простих елементів сортування буде найгіршим випадком буде масив відсортований за спаданням. Для алгоритму пошуку елемента в не відсортованому елементі worst case це коли елемент знаходиться в кінці масиву, або якщо елемента немає взагалі.

Найкращий випадок (best case) — повна протилежність worst case. Це найвдаліші вхідні дані. Наприклад це правильно відсортований масив з яким алгоритму сортування взагалі нічого робити не потрібно. У разі пошуку це випадок, коли алгоритм знаходить потрібний елемент з першого разу.

Середній випадок (average case) — найхитріший із трійки. Інтуїтивно зрозуміло що він знаходиться між best and worse case, але далеко не завжди зрозуміло де саме. Часто він збігається з worst case і майже завжди гірше best case, якщо best case не збігається з worst. Але зустрічаються випадки коли вони збігаються. Зазвичай середнім випадком вважають статистично усереднений результат: беруть алгоритм, прокручують його з різними даними, ставлять зведення результатів і дивляться навколо якої функції збереглися результати.

Види нотацій

1. Small o,
2. Big O,
3. Big Theta,
4. Big Omega,
5. Small omega.

Big O позначає верхню межу складності алгоритму. Це ідеальний варіант для позначення worst case.

Big Omega позначає нижню межу складності і її правильніше використовувати для пошуку best case.

Big Theta розташовується між O та Omega і показує точну функцію складності алгоритму з її допомогою правильніше шукати average case

Поширені складності алгоритмів

Складність залежить від алгоритму який ви оцінюєте, завжди може з'явитись якась змінна яку необхідно врахувати у функції Big O. Перша складність це залежність $O(1)$. Це означає що обчислювальна складність алгоритму не залежить від вхідних даних. Однак це не означає, що

алгоритм виконується за одну операцію, або вимагає дуже мало часу.

Приклад 1: у нас є масив зі 5 чисел і нам треба отримати перший елемент

```
val nums = arrayOf(1, 2, 3, 4, 5)
val firstNumber = nums[0]
```

Питання: наскільки зросте кількість операцій із збільшенням розміру вхідних параметрів.

Відповідь: навіть якщо масив складатиметься з 1000 елементів нам потрібна буде лише одна операція

Приклад 2: додавання двох чисел. Функція завжди виконує константу кількість операцій.

Приклад 3: визначити розмір масиву. Функція завжди виконує константу кількість операцій.

Наступна складність — лінійна $O(n)$ означає що складність алгоритму лінійно зростає із збільшенням вхідних даних інакше кажучи подвоєння подвоїть і час на виконання алгоритму. Такі алгоритми легко впізнати по наявності циклу, по кожному елементу масиву

Приклад 1: Рекурсивна функція :

```
fun sum(n: Int): Int {
    if (n == 1) return 1
    return n + sum(n - 1)
}
```

Приклад 2: Лінійна функція із циклу

```
fun pairSumSequence(n: Int): Int {
    var sum = 0
    for (i in 0 until n) {
        sum += pairSum(i, i + 1)
    }
    return sum
}

fun pairSum(a: Int, b: Int) = a + b
```

Відповідь: чим більше n тим більше разів виконується цикл

Приклад 3:

```
fun sum(nums: List<Int>) {
    var sum = 0
    var product = 1

    for(num in nums) {
        sum += num
    }

    for(num in nums) {
        product *= num
    }
}
```

```

    }

    println(sum, product)
}

```

у функції є два послідовних цикли for два з яких проходять масив довжиною циклу $O(n + n) = O(n)$.

Логарифмічна – $O(\log n)$ Означає що складність алгоритму зростає логарифмічно. Тобто це такий алгоритм де на кожній операції береться половина операцій. До логарифмів з такою складністю відносять алгоритми типу розділяй і володарюй наприклад бінарний пошук.

Наступна складній Лінеарифметична або лінеаризована - $O(n \cdot \log n)$. Означає що подвоєння типу даних трохи більше одинична

Наступна складність квадратична $O(n^2)$, $O(n^2)$ — підвищує час виконання вчетверо. Наприклад при збільшенні даних в 10, то час збільшиться приблизно в 100 разів. Якщо алгоритм має квадратичну складність це має привід переглянути необхідність використання даного алгоритму, але інколи цього не уникнути. Такі алгоритми легко впізнати за складеними циклами.

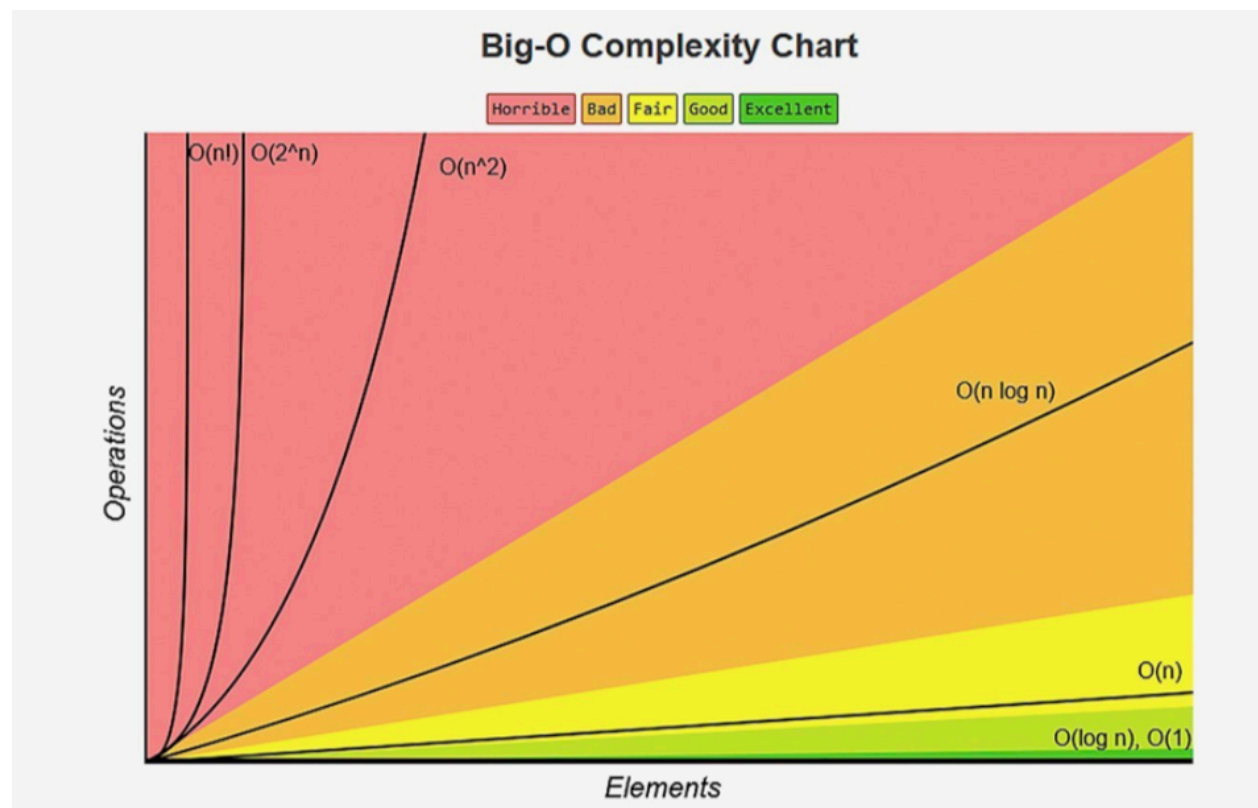
Приклад 1:

```

fun printPairs(nums: List<Int>) {
    for (i in nums) {
        for (j in nums) {
            println(nums[i], nums[j])
        }
    }
}

```

У функції є цикл у циклі кожен із них проходить цикл довжиною в n , отже складність буде $O(n * n) = O(n^2)$. Продемонструвати як відрізняються швидкості виконання різних функцій можна показати на графіках



Структури даних

Структура даних це сховище де ми тримаємо дані для подальшого використання. Вона представляє собою множину елементів даних та множину зв'язків між ними. Тобто структура даних — програмна одиниця, що дозволяє зберігати та обробляти множину однотипних та/або логічно пов'язаних даних за допомогою обчислювальної техніки. Кожна структура даних має свій інтерфейс який виконує такі функції як додавання, пошук, зміна та видалення даних. Структура даних є реалізацією абстрактного типу даних.

Існує два способи представлення структури даних — фізична та логічна

Структури даних, що розглядаються без результату їх представлення в машинній пам'яті називаються абстрактною або логічною структурою.

Поняття фізична структура даних відповідає способу фізичного представлення даних у пам'яті машини та називається ще структурою зберігання, внутрішньою структурою, або структурою пам'яті. Для точного опису абстрактних структур даних використовуються системи формальних позначень — мови програмування. Поняття фізичної структури даних можна звести до схеми організації інформації пам'яті комп'ютера. Де структурам даних відповідають структури реєстрів, алгоритмам ставляться електронні схеми.

Структури даних можна поділити на

- 1) прості (базові, примітивні)
- 2) інтегровані (композитні, складні)

Простими називаються такі структури даних, які не можуть бути розділені на прості частини, тобто які є неподільними одиницями. З точки зору фізичної структури в даній машинній архітектурі та системі програмування завжди відомо наперед розмір даного простого типу і його структура розміщення в пам'яті.

Інтегрованими називаються такі структури даних складовими частинами яких є інші структури даних. Інтегровані структури даних конструюються програмістом з використанням засобів інтеграції даних ,як представляються мовами програмування.



Ознака за розташуванням елементів у пам'яті:

-лінійні структури, які поділяються на послідовні рядки, масиви, с теки, черги; та довільні, однозв'язані та багатозв'язкові черги

-нелінійні структури — багатозв'язкові списки, дерева, графи

Ознака за змінністю, за можливістю змінити кількість елементів або зв'язків між елементами.

Статичні, напів статичні та динамічні

Прості базові структури

Стандартні типи даних часто називають арифметичними, оскільки їх можна використовувати в арифметичних операціях для опису основних типів визначено основні слова: основний, символьний, логічний, дійсний.

Ціле число в бінарному коді — внутрішнє представлення дійсного типу складається з двох частин мантий і порядку. Результати логічного типу отримуються при порівнянні даних будь-якого типу і можуть приймати тільки значення true or false(0) значенням символьного типу є символи з деякої наперед визначеної множини. Цією множиною є таблиця ASCII, що складається з 256 різних символів таке значення символьного типу займає в пам'яті 1 байт. Іншою множиною є код unicode де кожен символ кодується двома байтами, над арифметичними типами можливі 4 основні операції створення, знищення, вибір, оновлення.

Логічні операції над основними типами — додавання, віднімання, множення, ділення. Ще одна логічна операція над типами, =, !=, >, <. Перерахований тип визначається програмістом в ньому програміст перераховує всі значення які може приймати змінна цього типу.

Такий приклад

```
enum { CLUBS, DIAMONDS, HEARTS, SPADES } cardsuit;
```

означає що змінна типу cardsuit може примати одне з значень в дужках.

Інтервалі типи даних (типи діапазону), є підмножиною можливих значень стандартних типів даних int та char і застосовуються для зручнішого представлення інформації в програмі. Його опис зможе виконуватись таким чином:

<Назва змінної> = <початкова знач.>..<кінцева знач.>

Тип покажчик представляє собою адресу комірки пам'яті. При програмуванні на низькому рівні робота з адресами складає значну частину програм. При вирішенні прикладних задач з використанням мов високого рівня покажчик використовують при: необхідності представляти одну й ту саму ж ділянку пам'яті як дані різної логічної структури. При роботі з різними логічними типами даних. У C++ розрізняють три види покажчиків: на об'єкт, функцію, пустий тип. І можуть бути типізованими, та не типізованими. Коли йде мова про типізовані покажчики правильніше говорити про цілу сім'ю типів: покажчик на ціле, покажчик на символ. Не типізований показник використовується для представлення адреси, за якою містяться дані невідомого типу і можуть використовуватись тільки за збереженням адреси, звертання до якої неможливе, основними операціями в яких беруть участь покажчики, це присвоєння, отримання адреси, вибір. В C++ також доступні операції адресної арифметики які застосовуються в системному програмуванні.

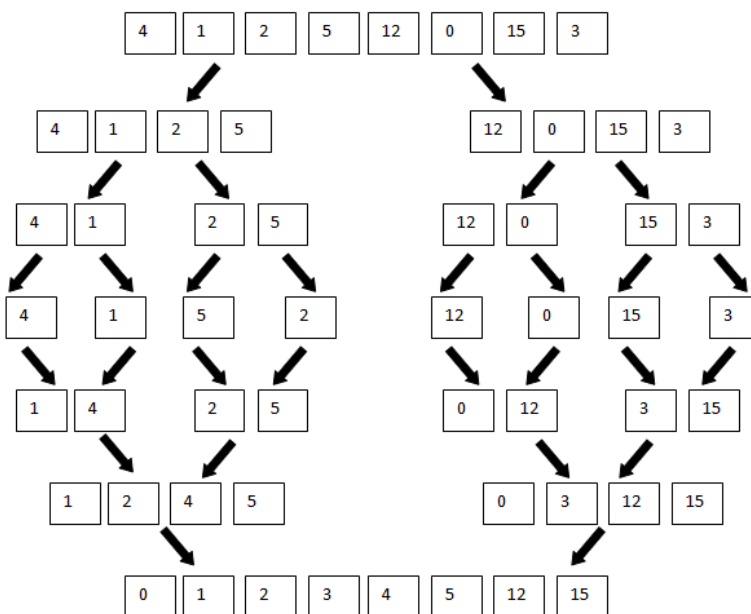
Статичні структури

Статичні структури відносяться до складу структур

сортування злиттям

В основу структури закладена стратегія складай і володарюй. В сортуванні пошарним злиттям вхідна множина розглядається як послідовність множин кожна з яких складається з єдиного

елемента і отже є впорядкованою на першому проході дві сусідніх елементні множини зливаються в одну двоелементну впорядковану множину. На другому проході двоелементні множини зливаються в чотириелементні впорядковані множини. Процес збільшення кількості елементів в множинах триває до поки не буде отримана одна велика впорядкована множина. Злиття двох впорядкованих множин відбувається пересилкою ключів із впорядкованих вух множин у вихідну множину. Почергово здійснюються порівняння ключів з першої і другої впорядкованої множини. Менше значення ключа записується у вихідну множину, далі збільшується на одиницю індекс елемента з впорядкованої множини з якої було взято елемент. Також збільшується індекс елемента у вихідній множини в кінці виконання цих дій у впорядкованих множинах можуть залишитися залишки в цих випадках частини впорядкованих величин що залишилися потрібно просто переслати у вихідну величину, алгоритми сортування злиття зазвичай мають порядок $O(N \cdot \log_2(N))$ і вимагають великої кількості пересилок, алгоритми злиття застосовуються в основному як складова частина зовнішнього сортування, роботу алгоритму сортування злиття масиву можна проілюструвати

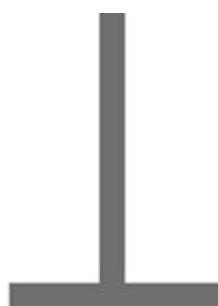


Метод розробки алгоритмів

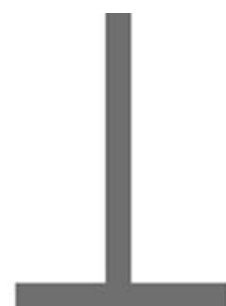
метод частково цілей. Метод рекурсивних алгоритмів є реалізацією методу часткових цілей. Сутність цього методу полягає у тому, що глобальна велика задача ділиться на окремі задачі. Кожну з поділених задач можна розв'язати вже відомим алгоритмом, який представляється значно легшою задачею ніж розв'язок великої задачі. Прикладом методу часткових цілей є задача сортування масиву прямим обміном алгоритм сортування зводиться до відшукування мінімуму в несортованій частині масиву та дописуванні його до сортованої частини класичний приклад методу часткових цілей є математична гра канойська вежа в задачі є піраміда із стрижнями різного розміру і дві пусті піраміди



A



B



C

Піраміду з осі А на вісь С зі збереженням порядку дисків користуючись віссю В як допоміжною, да додержуючись наступних правил 1.за один хід можна переставляти лише один диск з однієї осі на іншу, а не декілька 2. Забороняється класти більший диск на менший, тобто впорядкованість на кожній осі має зберігатися. Рекурсивний підхід до програмування можна застосувати, якщо розуміти вежу з n дисків, як n-1 дисків, таким чином можна зрозуміти алгоритм перестановки дисків в потрібному порядку.

Динамічне програмування

ідея динамічного програмування полягає в тому, що оптимальне рішення найчастіше можна знайти розглянувши всі можливі шляхи вирішення завдання і вибрати серед них найкраще. Робота динамічного програмування дуже нагадує рекурсію із запам'ятовуванням проміжних рішень, яку називають мемоізацією. Динамічні алгоритми ділять завдання на шматочки, та обчислюють їх по черзі нарощуючи рішення, тому динамічні алгоритми можна розглядати як рекурсію, що працює знизу на гору. Слід зазначити що такі алгоритми не вирішують одну й ту саму підзадачу двічі. При реалізації таких алгоритмів рішення дрібних підзавдань мають десь зберігатися, для багатьох реальних алгоритмів динамічного програмування такою структурою даних служить таблиця. Прикладами динамічного програмування можуть бути задачі про маршрут на прямокутному полі та задача про пошук найбільшої загальної послідовності.

Задача про маршрут на прямокутному полі: Дано прямокутне поле розміром $m \times n$ кожна комірка якого має певну вагу, що задається якимось натуральним числом можна здійснювати кроки вправо та вниз, необхідно потрапити з верхньої лівої комірки в праву нижню, і знайти маршрут з максимальною вагою. Загальна вага маршруту обчислюється як сума чисел всіх комірок, які були відвідані при розробці алгоритму треба вважати, що вага комірки першого етапу окрім першого елемента буде розраховуватися як сума ваги цього елемента і попереднього, тобто тут застосовується такий рекурентний вираз

$$a_{0,0} = a_{0,0}$$

$$a_{0,j} = a_{0,j} + a_{0,j-1}$$

$$a_{i,0} = a_{i,0} + a_{i-1,0}$$

Вага комірок з усіма номерами обчислюється як сума ваги поточної комірки і комірки з максимальною вагою, що взята з двох комірок, а саме на один рядок вище за поточну і на стовпець лівіше від поточної комірки. Вираз буде мати такий вигляд

$$a_{i,j} = a_{i,j} + \max(a_{i,j-1}, a_{i-1,j})$$

приклад

Розв'язати задачу про маршрут на прямокутному полі

3	2	7	4	12
11	8	9	1	5
8	6	7	13	10
2	9	4	7	3

В результаті виконаних дій отримаємо таку матрицю:

3	5	12	16	28
14	22	31	32	37
22	28	38	51	61
24	37	42	58	64

...

Наприклад можна з першого міста їхати у найближче, а звідти у найближче з тих, що залишилися. І т.д. з останнього міста треба буде повернутися назад. Такий варіант приймається першою вершиною, далі можна спробувати поліпшити цей варіант, якщо це вийде то це буде нова висота найкоротший цикл шукається за допомогою досить складних алгоритмів іншим прикладом є задача рюкзака у якій необхідно набрати набір предметів з обмеженим об'ємом. Жадібний алгоритм для задачі рюкзака вибирає предмети з найбільшою вартістю на одиницю ваги та додає їх до рюкзака до тих пір поки не буде досягнуто обмеження об'єму. Також до жадібних алгоритмів можна віднести алгоритм побудови найкоротшого шляху Дейкстри і алгоритм побудови стовбурного дерева мінімальною вартістю Крอสкала. Алгоритм найкоротшого шляху є жадібним у тому розумінні що він вибирає вершину найближчу до джерела серед тих, найкоротший шлях яких ще не відомий, алгоритм Кроскала обирає ребро з мінімальною вартістю із ребер які залишилися і не створюють цикл у більшості випадків жадібні алгоритми забезпечують локальний оптимум у той же час, як в цілому результат буде не оптимальним. Порівняно із жадібними алгоритмами динамічне програмування може знайти глобально-оптимальне рішення, але може бути більш складним у виконанні та вимагати більше обчислювальних ресурсів.

Дерево ухвалення рішень(дерево розв'язків)

Дерева ухвалення рішень використовують у випадку автоматичного аналізу великих масивів даних цей інструмент можна ефективно застосовувати для інтелектуального аналізу даних та передбачувальної аналітики цей інструмент можна ефективно застосовувати для інтелектуальної обробки даних та передбачувальної логіки, вирішуючи завдання пов'язані із кваліфікацією та регресією. Дерево рішень представляє собою ієрархічну дерево-подібну структуру, що складається з правила виду, (якщо..., то...) . За рахунок навчальної множини правила генеруються автоматично в процесі навчання на відміну від нейронних мереж дерева як аналітичні моделі простіші, тому що правила генеруються природною мовою. Наприклад: якщо реклама привела 1000 клієнтів, то вона налаштована добре. Правила генеруються за допомогою узагальнення безліччю окремих спостережень, які описують предметну область, тому їх називають індуктивними правилами, а процес навчання — індукцією дерев рішень. Кожен вузол дерева представляє один крок вирішення задачі. Кожна гілка в дереві представляє розв'язок який веде до більш повного рішення листки є остаточним

розв'язком, мета полягає в тому, щоб знати найкращий шлях від кореня дерева до листка при виконанні певних умов, ці умови значення поняття найкращий для шляху залежать від конкретної задачі, такі дерева можна застосовувати для моделювання таких ігор як шашки, хрестики-нолики. Кількість можливих ходів у грі буде відповідати кількості гілок у дереві. Наприклад при початку гри у хрестики-нолики, кореневий вузол відповідає початковій позиції коли дошка порожня. Перший гравець може виконати хід помістивши хрестик в одну з дев'яти клітинок дошки відповідно в дереві це буде відображено у вигляді дев'яти гілок, які виходять з коренневого вузла. Вузли на кінцях цих гілок відображають позиції після першого ходу. Тепер другий гравець може виконати хід в одну із восьми гілок, що залишились. Дерево гри навіть для такої простої гри росте дуже швидко, якщо воно буде продовжувати рости з таким темпом, то кожний наступний вузол у дереві матиме на одну гілку менше за попередню, то повне дерево гри матиме $9!$ веток. Багато варіантів розвитку гри. Скоротити кількість вузлів можна виключивши вузли які відповідають ходам, які заборонені правилами гри. Наприклад якщо гравець який ходив перший за три свої роди поставити хрестики у верхній лівій, верхній середній та верхній правій клітках то він виграє і гра закінчиться. Вузол який відповідає цій позиції не матиме нащадків, оскільки гра завершується на цьому кроці. Після вилучення всіх неможливих вузлів у дереві залишається біля четверті мільйона листів, але дерево все-ж залишається великим і пошук у ньому оптимального розв'язку методом повного перебору займає достатньо багато часу. Можна ще скоротити розмір цього дерева враховуючи симетричність деяких позицій, але це підходить лише для конкретної гри. Для складніших ігор таких як шахи, шашки дерева мають величезний розмір, якби під час кожного ходу в шахах гравець мав 16 різних варіантів то дерево гри мало б більше трильйона вузлів після 5 ходів кожного гравця. Для того щоб скоротити ще більше кількість вузлів для виконання пошуку в дереві гри потрібно мати можливість визначити вагу позицій на дошці наприклад у грі в хрестики-нолики для першого гравця більшу вагу мають позиції в яких три хрестики розташовані в ряд, оскільки при цьому він виграє. Вага тих же позицій для другого гравця мала тому що в цьому випадку він програє. Для кожного гравця можна присвоїти кожній позиції один з чотирьох вагових коефіцієнтів. Якщо коефіцієнт рівний чотирьом то це значить що гравець в цій позиції виграє, якщо трьома то з поточного положення на дошці не ясно хто з гравців виграє врешті решт. Двійка означає що позиція приводить до нічиєї, одиниця — в грає супротивник в при такому підході для пошуку розв'язку методом повного перебору можна використовувати міні-максну стратегію при якій робиться спроба мінімізувати максимальну вагу, яку може мати позиція для супротивника після наступного ходу це можна зробити визначивши максимально можливу вагу позиції для супротивника після кожного з своїх можливих ходів і потім вибрати позицію ходу з мінімальною вагою для супротивника, тобто алгоритм повинен обчислювати вагу позицій на дошці, перевіряючи всі можливі ходи. Для кожного ходу він повинен рекурсивно викликати себе щоб знайти вагу яку матиме нова позиція для супротивника, потім вибирається хід при якому вага отриманої позиції для супротивника буде як найменшою програмно для визначення ваги позиції на дошці потрібна функція яка рекурсивно викликає себе до тих пір поки не відбудеться одна з трьох подій. По-перше вона може дійти до позиції в якій гравець виграє в цьому випадку вона присвоює позиції ваговий коефіцієнт 4, що вказує на виграш гравця, що виконав останній хід, по—друге може знайти позиції. В якій жоден з гравців не може зробити наступний хід. Гра при цьому закінчується нічиєю. Тому тому функція присвоює тій позиції 2, і нарешті функція може досягти заданої максимальної глибини рекурсії в цьому випадку процедура присвоює позиції 3 що вказує на неможливість визначити переможця. Функція при цьому вибирає хід який має як найменшу вагу для супротивника. Завдання максимальної глибини рекурсії обмежує час пошуку в деревах розв'язку. Це особливо важливіше для складних ігор (шахи, шашки,) в яких пошук в дереві гри може продовжуватись майже вічно. Максимальна глибина пошуку також може задавати рівень майстерності програми, чим далі в перед програма зможе аналізувати шахи тим краще вона гратиме. В дереві гри використовується не лише міні-максна стратегія. Можна, наприклад, записати у програму початкові ходи. Які обрані експертами, наприклад хід у центральну клітку в хрестики-нолики. Тоді з програми можна виключити всі шляхи які не проходять через відповідну цьому ходу гілку а також можна врахувати той факт що програмі не потрібно виконувати пошук в дереві до того поки супротивник не зробить свій хід у цей момент і комп'ютер і і супротивник вибрали кожен свою гілку дерево яке залишилось стане набагато меншим і міститиме менше ніж $17=5040$ шляхів, якщо зменшити розмір дерева гри менше 5050 шляхів отже прорахувавши наперед всього один хід можна від четверті мільйона до 5040, аналогічно для можна записати відповіді на перші ходи якщо супротивник ходить першим. Є 9 варіантів першого ходу, отже має бути 9 варіантів ходів. При цьому програмі не потрібно проводити пошук деревом поки супротивник не зробить два ходи, а комп'ютер 1, тоді дерево гір міститиме менше ніж 6! Шляхів, тобто записано всього 9 ходів, а розмір дерева зменшується при цьому дуже сильно. Це ще один приклад просторово-часового компромісу. Використовування більшої кількості пам'яті зменшує час необхідний для пошуку в дереві гри. Тобто програма записує наперед декілька початкових ходів на початку гри буде працювати дуже швидко.